SPM Module 1: Vectorization of the Partition Mapping Kernel

Teja Karuku
Matriculation: 724955

March 2026

1 Introduction

The main goal of this project is to investigate Data-Level Parallelism (DLP) by enhancing a core systems task: assigning a massive array of keys to designated partitions. Within modern High-Performance Computing (HPC) and database architectures, optimized data partitioning serves as a vital foundation for distributed operations, including parallel aggregations and Radix Hash Joins.

By adopting Single Instruction, Multiple Data (SIMD) paradigms, a single processor core can compute multiple data elements at the same time, leading to a massive boost in instruction throughput. This document outlines the design, validation, and performance benchmarking of a partitioning kernel across three iterative phases: a standard scalar baseline, a compiler-driven auto-vectorized implementation, and a manually tuned version using Advanced Vector Extensions 2 (AVX2) intrinsics.

2 Experimental Setup and Hardware Constraints

To guarantee the accuracy of our performance measurements, all testing took place on a designated compute node (`node09`). The experimental parameters were fixed at $N = 50,000,000$ elements distributed across $P = 256$ partitions.

Selecting $N = 50,000,000$ 64-bit integers was a calculated decision. This configuration generates an input array of precisely 400 Megabytes (MB) alongside a 200 MB output array. Because the combined 600 MB memory footprint greatly exceeds the limits of standard L1, L2, and L3 processor caches, the CPU must stream data directly from the main memory (RAM). This physical limitation ensures our benchmarks reflect sustained system memory bandwidth, rather than artificial, in-cache execution speeds.

3 Algorithmic Design and Complexity

3.1 Data Generation and Verification

To accurately evaluate the mapping logic in isolation, the execution is divided into two distinct steps. Initially, the input array is populated deterministically using a Mersenne Twister pseudo-random number generator (`std::mt19937_64`) seeded with a constant value (12345). Performing this step outside the benchmarking timer guarantees that the computationally heavy random number generation does not distort the partitioning metrics. A checksum verification routine ensures that all three implementations yield identical mathematical results.

3.2 Power-of-Two Masking Optimization

The partition count was fixed at $P = 256$. The key architectural decision here is ensuring that P remains a power of two. Traditional SIMD instruction sets, including AVX2, lack a dedicated 64-bit integer modulo (`%`) command, and standard integer division is highly inefficient (frequently consuming 10 to 20 CPU cycles).

By keeping P as a power of two, the standard modulo expression (`key % P`) can be mathematically replaced by a bitwise AND calculation (`key & (P - 1)`). Within the implementation, a masking variable is created as `P - 1`. This optimization converts a costly division task into a rapid, single-cycle bitwise operation, maximizing potential throughput. The time complexity for this phase is exactly $O(N)$, with a space complexity of $O(N)$ required for array storage.

3.3 Pointer Aliasing (`__restrict__`)

A major obstacle in C++ compiler auto-vectorization is "pointer aliasing"—the risk that memory addresses for input and output arrays might overlap, preventing the compiler from confidently parallelizing the instructions. To mitigate this, the `__restrict__` qualifier was strategically applied to both the `keys` and `part_id` pointers in the function signatures. This serves as a formal guarantee to the compiler that the memory regions are strictly independent.

4 GCC Auto-Vectorization Evidence

To establish a performance baseline, the C++ source was compiled under GCC with vectorization explicitly disabled (`-fno-tree-vectorize`). For the auto-vectorized variant, the compiler was invoked using the `-ftree-vectorize` and `-mavx2` flags. We generated a diagnostic log using the `-fopt-info-vec` parameter.

Compiler Evidence and Dependency Analysis:
The resulting diagnostic output verified that GCC successfully executed loop unrolling and dependency checks to vectorize the primary mapping loop. Advanced static analysis allows modern compilers to verify the absence of loop-carried dependencies. By proving that a given iteration does not rely on the output of a prior one, GCC safely translates the scalar bitwise logic into 256-bit YMM register instructions. The `-fopt-info-vec` flag is an essential tool in this process, offering clear feedback that the compiler did not abandon vectorization due to suspected aliasing or complex branching.

```
[[t.karuku@spmln spm_module01]$ cat vec_report.txt
 Mapper.cpp:19:26: optimized: loop vectorized using 32 byte vectors
 Mapper.cpp:19:26: optimized: loop vectorized using 16 byte vectors
 /opt/ohpc/pub/compiler/gcc/12.2.0/include/c++/12.2.0/bits/random.tcc:412:42: optimized: loop vectorized using 32 byte vectors
 /opt/ohpc/pub/compiler/gcc/12.2.0/include/c++/12.2.0/bits/random.tcc:412:42: optimized: loop vectorized using 16 byte vectors
 /opt/ohpc/pub/compiler/gcc/12.2.0/include/c++/12.2.0/bits/random.tcc:404:32: optimized: loop vectorized using 32 byte vectors
 Mapper.cpp:61:26: optimized: loop vectorized using 32 byte vectors
 Mapper.cpp:19:26: optimized: loop vectorized using 32 byte vectors
```

Figure 1: Terminal output demonstrating the GCC vectorization report for the hot loop.

As illustrated in Figure 1, the compiler logs the phrase: `optimized: loop vectorized using 32 byte vectors`. Because 32 bytes translates to exactly 256 bits, this provides undeniable proof that the scalar C++ logic was successfully scaled to utilize the entire width of the AVX2 hardware registers.

5 Manual AVX2 Intrinsics Implementation

To explicitly vectorize the mapping routine, the AVX2 instruction set was leveraged via the `<immintrin.h>` library. Given that AVX2 registers span 256 bits and the data keys are 64-bit

integers, this approach computes chunks of 4 keys concurrently in a single cycle.

Implementation Details:

- **Mask Broadcast:** The bitwise mask (`P - 1`) is duplicated across all four 64-bit segments of a 256-bit register utilizing `_mm256_set1_epi64x`.
- **Memory Alignment and Loading:** Inside the primary loop, 4 contiguous 64-bit keys are fetched from RAM into a `__m256i` register. It is vital that the `_mm256_loadu_si256` intrinsic is applied. The "u" indicates an unaligned memory load. Although aligned loads (`load_si256`) offer marginal historical speed benefits, unaligned loads safeguard against segmentation faults in cases where the `std::vector` allocator fails to align memory blocks to strict 32-byte boundaries.
- **Parallel Execution:** The mapping logic is executed simultaneously on all 4 keys via the `_mm256_and_si256` intrinsic.
- **Extraction:** AVX2 does not offer a native instruction to effortlessly downcast and flush 64-bit vector lanes into a sequential 32-bit array. Consequently, results must be extracted individually using `_mm256_extract_epi64`, cast to standard 32-bit integers, and stored in order.
- **Tail Handling (Amdahl's Law):** A traditional scalar loop follows the vectorized segment to cleanly process any leftover elements in the event that N is not an exact multiple of 4.

Performance Summary Analysis:

Transitioning from manual intrinsic coding to empirical testing lets us evaluate the real-world impact of SIMD. By comparing our hand-tuned hardware approach against the compiler's automated methods, we can assess whether the computational overhead of vector extraction and memory management negates the raw speed of AVX2 execution units. The figures below display the terminal benchmarks for all three compiled programs.

```
t.karuku@node09:~/spm_module01$ ./BenchNonVec
Allocating memory for 50000000 elements...
Generating input keys deterministically...

=== Running Benchmark: Plain C++ ===
Median Execution Time: 0.0418218 s
Standard Deviation:    4.16517e-05 s
Throughput:            1.20e+09 elements/s
Verification Checksum: 6375328077
t.karuku@node09:~/spm_module01$
```

nonvec

```
t.karuku@node09:~/spm_module01$ ./BenchVec
Allocating memory for 50000000 elements...
Generating input keys deterministically...

=== Running Benchmark: Plain C++ ===
Median Execution Time: 0.0403613 s
Standard Deviation:    0.000306418 s
Throughput:            1.24e+09 elements/s
Verification Checksum: 6375328077
t.karuku@node09:~/spm_module01$
```

vec

```
t.karuku@node09:~/spm_module01$ ./BenchAVX2
Allocating memory for 50000000 elements...
Generating input keys deterministically...

=== Running Benchmark: Plain C++ ===
Median Execution Time: 0.0402155 s
Standard Deviation:    0.000287488 s
Throughput:            1.24e+09 elements/s
Verification Checksum: 6375328077

=== Running Benchmark: AVX2 Intrinsics ===
Median Execution Time: 4.02e-02 s
Standard Deviation:    5.57e-05 s
Throughput:            1.24e+09 elements/s
Verification Checksum: 6375328077
t.karuku@node09:~/spm_module01$
```

avx2

Figure 2: Empirical benchmarking outputs for Plain C++, Auto-Vectorized GCC, and Manual AVX2 Intrinsics.

6 Performance Evaluation and Roofline Analysis

Methodology:
To minimize the impact of operating system noise, the kernel was run 20 times for each implementation. We logged the median execution time, standard deviation, and overall throughput. Every version successfully generated the identical verification checksum (6375328077), confirming functional correctness.

Results Summary:

- **Baseline (Plain C++):** Median Time = 0.0418 s, Throughput = 1.20e+09 elements/s
- **Auto-Vectorized (GCC):** Median Time = 0.0403 s, Throughput = 1.24e+09 elements/s
- **Manual AVX2 Intrinsics:** Median Time = 0.0402 s, Throughput = 1.24e+09 elements/s

6.1 The Roofline Bottleneck

Although vectorization did provide a measurable performance lift over the scalar baseline, the overall speedup remains highly restricted (yielding an approximate 4% enhancement). To comprehend why the manual AVX2 code fell short of the theoretical 4x speedup, we evaluate the program through the lens of the **Roofline Model** for hardware performance.

This specific application exhibits remarkably low Arithmetic Intensity. Operating at a peak throughput of 1.24 billion elements per second, the routine ingests 400 MB of data and writes out 200 MB in merely 0.04 seconds. This translates to a continuous memory bandwidth demand of approximately 15 Gigabytes per second (GB/s).

Because the core mathematical operations are extremely trivial (consisting of a single hardware bitwise AND), the processor finishes its arithmetic long before the RAM can supply or store the data across the memory bus. Consequently, the AVX2 execution units spend a vast majority of their operational cycles stalled, idling while cache lines are populated. Ultimately, the application is constrained by the "memory bandwidth roof" of `node09`.

7 Conclusion

This assignment successfully illustrated the principles of Data-Level Parallelism. Although the manual AVX2 implementation verified that 256-bit registers can be successfully and explicitly programmed, the benchmarking data underscores a core principle of HPC optimization: accelerating the CPU instruction pipeline offers severely diminishing returns when an application is inherently constrained by main memory bandwidth.